

Entrega Final — Desenvolvimento de aplicações Java com Spring Boot

Aluno: Jairo Williams Guedes Lopes Neto **Disciplina:** Desenvolvimento de aplicações Java com Spring Boot **Data:** 31/08/2026

Repositório

https://github.com/JairoNetoDev/permission_saas

A versão final corresponde à tag `etapa-4`. As quatro tags são os marcos de cada competência:

Tag	Competência avaliada
<code>etapa-1</code>	Modelo Orientado a Objetos
<code>etapa-2</code>	Collections + Service + armazenamento em memória
<code>etapa-3</code>	Spring Boot + API REST
<code>etapa-4</code>	Spring Data JPA + Banco de Dados — versão final

```
git clone https://github.com/JairoNetoDev/permission_saas.git
cd permission_saas
git tag -l          # etapa-1 etapa-2 etapa-3 etapa-4
git checkout etapa-4
```

Sobre o projeto

Permission SaaS — um SaaS de gerenciamento de permissões por projeto. Um cliente se cadastra, assina um plano (pagamento simulado) e recebe uma **ApiKey**. Sistemas externos usam essa ApiKey para validar, em um único endpoint, se uma requisição pode acessar determinada rota com determinado cargo.

Este é o **projeto guarda-chuva da minha Pós-Graduação**: cada disciplina evolui o mesmo código em vez de começar do zero. A disciplina anterior (Clean Code e Padrões de Projeto) entregou os módulos `shared`, `identity`, `billing` e `permission`. **Esta disciplina acrescentou os módulos `project` e `audit`, a persistência JPA de ambos e a regra real de validação de permissão**. A separação está registrada em `docs/ARCHITECTURE.md` e nos planos de cada disciplina.

Arquitetura: monolito modular (Spring Modulith) com quatro camadas por módulo — `domain`, `application`, `infrastructure`, `api`. As fronteiras entre módulos são verificadas por teste automatizado (`ApplicationModulesIntegrationTests`), que quebra o build se alguma for violada.

Stack: Java 21 · Spring Boot · Spring Data JPA · PostgreSQL 16 · Flyway · Spring Modulith · Docker Compose · Maven.

Como executar

Instruções completas no `README.md` do repositório. Resumo:

```
cp .env.example .env           # variáveis de ambiente
docker compose up --build      # sobe aplicação + PostgreSQL
curl http://localhost:8080/ping
```

- **Banco:** `permissions_saas` / usuário `saas` / senha `saas123` / porta `5432`. O schema é criado pelo Flyway (`src/main/resources/db/migration/V1..V9`); `spring.jpa.hibernate.ddl-auto` é `validate`, então o mapeamento JPA é conferido contra as migrations a cada inicialização.
- **Swagger UI:** <http://localhost:8080/swagger-ui.html>
- **Coleção Postman:** `docs/postman/permission-saas.postman_collection.json` — 42 requisições, 60 asserções, executável com `newman run`.

```
./mvnw verify                  # 22 testes de unidade + 9 de integração
```

Onde encontrar cada coisa no código

O quê	Onde
Relacionamento um-para-muitos	<code>ProjectJpaEntity</code> (@OneToMany para cargos e rotas), <code>RoleJpaEntity</code> e <code>RouteJpaEntity</code> (@OneToMany para as concessões, @ManyToOne para o projeto), <code>RoleRouteJpaEntity</code> (@ManyToOne duplo, para cargo e para rota)
Herança entre entidades persistentes	<code>AuditEventJpaEntity</code> (abstrata) → <code>PermissionCheckEventJpaEntity</code> e <code>ProjectLifecycleEventJpaEntity</code> , estratégia <code>SINGLE_TABLE</code> com <code>@DiscriminatorColumn</code>
Atributos e seus tipos	texto (<code>name</code> , <code>path</code> , <code>reason</code>), inteiro (<code>maxRoles</code>), real (<code>BigDecimal price</code> , <code>double durationMs</code>), booleano (<code>isActive</code> , <code>granted</code>), data (<code>OffsetDateTime createdAt</code> , <code>grantedAt</code> , <code>revokedAt</code> , <code>occurredAt</code>). Glossário completo em <code>docs/DOMAIN.md</code>
<code>toString</code> com o relacionamento	<code>Project.toString()</code> lista cargos e rotas; <code>Role.toString()</code> lista as concessões
Entidades JPA	11 classes <code>@Entity</code> : <code>Project</code> , <code>Role</code> , <code>Route</code> , <code>RoleRoute</code> , <code>AuditEvent</code> (+ 2 subclasses), <code>Client</code> , <code>Plan</code> , <code>Subscription</code> , <code>ApiKey</code>
Camada repository	<code>JpaProjectRepository</code> , <code>JpaAuditEventRepository</code> , <code>JpaClientRepository</code> , <code>PlanJpaRepository</code> , <code>SubscriptionJpaRepository</code> e <code>ApiKeyJpaRepository</code> , todas estendendo <code>JpaRepository</code> , com consultas derivadas
Camada de serviço usando os repositories	Os use cases injetam a porta declarada no <code>domain</code> (<code>ProjectRepository</code> , <code>AuditEventRepository</code>), implementada pelo adapter JPA. Foi essa inversão que permitiu trocar o <code>Map</code> pelo banco sem alterar nenhum use case, controller ou mapper
Endpoints REST	<code>/clients</code> , <code>/plans</code> , <code>/subscriptions</code> , <code>/projects</code> (com os sub-recursos <code>/roles</code> , <code>/routes</code> e as concessões de rota a cargo), <code>/validate-permission</code> , <code>/audit-events</code> — todos detalhados em <code>docs/API.md</code>
Testes dos endpoints	Coleção Postman com 42 requisições e 60 asserções (<code>newman run</code>), mais 22 testes de unidade e 9 de integração em <code>./mvnw verify</code>
Documentação dos endpoints	Swagger UI em <code>/swagger-ui.html</code> e <code>docs/API.md</code>
Migrations do banco	<code>src/main/resources/db/migration/V1</code> a <code>V9</code>
Regras de negócio e invariantes	<code>docs/DOMAIN.md</code>
Decisões de arquitetura e seus custos	<code>docs/ARCHITECTURE.md</code> (7 ADRs)

Sobre o `Map` em memória e os loaders de arquivo texto

Eles existiram nas etapas 1 a 3 e **foram removidos do código final**, quando a persistência em banco entrou. A remoção segue o enunciado: *"Não é necessário manter implementações antigas artificialmente na versão final. O projeto deve evoluir normalmente, utilizando o controle de versão para preservar seu histórico."*

O histórico está preservado nas tags e pode ser inspecionado sem trocar de branch:

```
# Map simulando a base de dados, e um serviço que o gere
git show etapa-3:src/main/java/com/saas/permissions/project/infrastructure/InMemoryProjectRepository.java
git show etapa-2:src/main/java/com/saas/permissions/project/application/SearchProjectsUseCase.java

# Loader lendo os arquivos texto e montando o relacionamento um-para-muitos
git show etapa-3:src/main/java/com/saas/permissions/project/infrastructure/ProjectFileLoader.java
git show etapa-3:src/main/resources/data/projects.txt
git show etapa-3:src/main/resources/data/roles.txt
git show etapa-3:src/main/resources/data/routes.txt
```

Os motivos da remoção estão registrados em `docs/ARCHITECTURE.md`, ADR-005.

Evolução por etapa

Etapa 1 — Orientação a Objetos. Modelagem de `Project`, `Role` e `Route` com invariantes no próprio agregado (limite de cargos do plano, unicidade de cargo, unicidade de método+caminho) e da hierarquia `AuditEvent` → `PermissionCheckEvent` / `ProjectLifecycleEvent`. Loaders lendo os três arquivos texto e montando o grafo 1-N pelos métodos de domínio, de modo que o seed passa pelas mesmas validações.

Etapa 2 — Collections e Serviços. Porta `ProjectRepository` no domain e adapter `InMemoryProjectRepository` encapsulando um `ConcurrentHashMap`. Use cases de escrita e de consulta com Streams e lambdas: filtro por trecho de nome, filtro de rotas por método HTTP, ordenações e resumo com `map` / `reduce`.

Etapa 3 — API REST. CRUD completo em `/projects` mais os sub-recursos, com `201 + Location`, `204`, `400`, `404` e `409`; Bean Validation nos DTOs; serialização do 1-N sem referência circular; trilha de auditoria alimentada por evento de aplicação (padrão Observer entre módulos); coleção Postman versionada.

Etapa 4 — Spring Data JPA. Substituição do `Map` pela persistência real, sem alterar nenhum controller ou use case — só o adapter por trás da porta. Entram o relacionamento `@OneToMany` / `@ManyToOne` com cascata, a herança `SINGLE_TABLE` do `audit`, as consultas derivadas e a entidade associativa `RoleRoute`, que finalmente permite responder à pergunta central do domínio: **o cargo X pode acessar a rota Y no projeto Z?**

`RoleRoute` guarda `grantedAt` e `revokedAt`: revogar **fecha** o registro em vez de apagá-lo, o que permite à auditoria responder *quando* um cargo deixou de ter acesso a uma rota. Um índice único parcial garante no máximo uma concessão ativa por par, sem impedir o empilhamento do histórico.

Documentação

Toda no diretório `docs/` do repositório:

Arquivo	Conteúdo
DOMAIN.md	Glossário do domínio: entidades, atributos, invariantes e regras
ARCHITECTURE.md	Mapa dos módulos, regras entre camadas e 7 ADRs registrando as decisões e seus custos
API.md	Todos os endpoints: método, caminho, corpo, respostas, erros e exemplos curl
PATTERNS.md	Padrões GoF aplicados: onde estão, por que foram escolhidos, como estender
TEST-ARCHITECTURE.md	Camadas de teste, convenções e o que cada teste de integração protege
postman/	Coleção de requisições usada nos testes
desenvolvimento_de_aplicacoes_java_com_spring_boot/PLAN.md	Planejamento e diário desta disciplina

Fontes

- Documentação oficial do Spring Boot, Spring Data JPA, Spring Modulith e Hibernate.
- Documentação do PostgreSQL e do Flyway.
- Ferramenta de IA: Claude, da Anthropic. O registro do uso ao longo da disciplina está em docs/desenvolvimento_de_aplicacoes_java_com_spring_boot/PLAN.md.